

# Lekce 16: Úvod do Cypress pluginů a rozšíření (zkráceně)

## Cíle

- Prozkoumat roli Cypress pluginů a rozšíření při rozšiřování možností testování.
- Naučit se instalovat, konfigurovat a využívat populární Cypress pluginy.
- Pochopit osvědčené postupy pro výběr a údržbu konfigurace pluginů, abyste udrželi svůj testovací balík efektivní a snadno spravovatelný.

## Stručný obsah

### A. Cypress Pluginy

#### 1. Co jsou Cypress pluginy?

- **Definice:**

Cypress pluginy jsou moduly nebo knihovny, které rozšiřují základní funkčnost Cypressu. Umožňují vám přizpůsobit a rozšířit, jak jsou testy vykonávány.

- **Účel:**

- Rozšířit možnosti Cypressu za jeho vestavěné funkce.
- Integrovat s externími nástroji a frameworky.
- Automatizovat úkoly jako reportování, testování přístupnosti a správa stahování souborů.

## Moduly a knihovny ve vývoji v Node.js

### Moduly:

- **Definice:**

V Node.js je modul samostatný kus kódu – často jeden nebo několik souborů, které mohou exportovat funkcionalitu (například funkce, objekty, třídy) a importovat funkcionalitu z jiných modulů. Moduly pomáhají organizovat kód do znovupoužitelných a udržovatelných částí.

- **Jak moduly fungují:**

Node.js ve výchozím nastavení používá modulární systém CommonJS (pomocí `require()` a `module.exports`), nyní ale také podporuje ES moduly (pomocí `import` a `export`). Každý modul má svůj vlastní rozsah, což znamená, že proměnné a funkce definované v jednom modulu nejsou přístupné v jiném, pokud nejsou explicitně exportovány.

- **Typy modulů:**

- **Vestavěné moduly:**

Node.js obsahuje mnoho vestavěných modulů (např. `fs` pro operace se souborovým systémem, `http` pro vytváření serverů, `path` pro manipulaci s cestami).

*Příklad:*

```
const fs = require('fs');
```

- **Třetí strany:**

Moduly publikované na npm (Node Package Manager), které můžete instalovat a používat ve svých projektech (např. `express`, `lodash`, `axios`).

*Příklad:*

```
const express = require('express');
```

- **Vlastní moduly:**

Moduly, které sami napíšete pro specifické funkce vaší aplikace.

*Příklad:*

```
// math.js
function add(a, b) { return a + b; }
module.exports = { add };
```

## **Knihovny:**

- **Definice:**

Knihovna je kolekce modulů navržená tak, aby nabízela soubor funkcionalit. Knihovny mohou urychlit vývoj opětovným použitím běžné logiky (například frameworky jako Express pro vývoj webových aplikací).

- **Rozdíl mezi modulem a knihovnou:**

- **Modul:** Typicky jedna jednotka kódu se specifickou odpovědností.
  - **Knihovna:** Kolekce modulů poskytující širší funkcionalitu.

- **Příklady v Node.js:**

- **Express:** Knihovna pro tvorbu webových serverů a API.
  - **Lodash:** Uživatelská knihovna nabízející mnoho pomocných funkcí pro manipulaci s daty.

## **Proč jsou tyto pluginy užitečné:**

- **Zvyšují možnosti testování:**

Rozšiřují základní funkčnost Cypressu a umožňují provádět úkoly, které by jinak vyžadovaly

vlastní kód.

- **Zlepšují zážitek vývojáře:**

Díky dodatečným příkazům a lepšímu reportování je ladění a údržba testů jednodušší.

- **Zajišťují spolehlivost testů:**

Nahrazováním síťových požadavků, simulací skutečných uživatelských událostí nebo řízením testovacích sezení pomáhají pluginy snižovat nestabilitu testů.

- **Podpora specifických testovacích potřeb:**

Ať už potřebujete testování přístupnosti, vizuální regresní testy nebo pokročilé ověřování, pravděpodobně existuje plugin, který vaše požadavky řeší.

## 2. Instalace a konfigurace základních Cypress pluginů:

- **Proces instalace:**

- Pluginy se obvykle instalují pomocí npm (např. `npm install cypress-axe --save-dev`).

- **Konfigurace:**

- Po instalaci se pluginy často konfigurují v souboru `cypress.config.js` nebo v souborech ve složce `cypress/plugins/`.

- **Příklad – Instalace cypress-axe pro testování přístupnosti:**

```
npm install cypress-axe --save-dev
```

- Poté ve vašem `cypress/support/commands.js` nebo `cypress/support/index.js` importujete a inicializujete plugin:

```
import 'cypress-axe';
```

- Nyní můžete ve svých testech používat příkazy jako `cy.injectAxe()` a `cy.checkA11y()`.

## 3. Přehled populárních pluginů:

Ekosystém Cypressu zahrnuje mnoho pluginů, které rozšiřují jeho funkčnost. Zde je 15–20 důležitých nebo populárních pluginů s krátkým popisem:

### i. **cypress-axe:**

- Integruje testovací engine axe pro přístupnost se Cypressem, pro automatizované kontroly přístupnosti.

### ii. **cypress-grep:**

- Umožňuje filtrovat a spouštět testy podle tagů či vzorů, což zjednodušuje spouštění dílčích částí testovací sady.

### iii. **cypress-file-upload:**

- Poskytuje příkazy pro simulaci nahrávání souborů v testech.

### iv. **cypress-real-events:**

- Simuluje nativní události prohlížeče (např. reálné pohyby myši nebo klávesové události) pro realističtější testy.

v. **cypress-mochawesome-reporter:**

- Generuje krásné a interaktivní testovací reporty pomocí nástroje Mochawesome.

vi. **cypress-cucumber-preprocessor:**

- Umožňuje psát testy ve formátu Gherkin (Cucumber), což je užitečné pro přístup Behavior-Driven Development (BDD).

vii. **cypress-wait-until:**

- Poskytuje vlastní příkaz pro čekání, dokud nebude splněna podmínka – užitečné pro asynchronní chování.

viii. **cypress-failed-log:**

- Zaznamenává dodatečné detaily (například síťové požadavky) při selhání testu pro snadnější ladění.

ix. **cypress-shadow-dom:**

- Zlepšuje podporu testování komponent používajících Shadow DOM díky vlastním příkazům pro jeho traversování.

x. **cypress-select-tests:**

- Pomáhá vybrat a spustit konkrétní testy na základě tagů či pojmenování.

xi. **cypress-plugin-tab:**

- Simuluje události klávesy Tab pro lepší testování přístupnosti ovládání klávesnicí.

xii. **cypress-ntlm-auth:**

- Poskytuje podporu pro NTLM autentizaci – užitečné ve firemních prostředích.

xiii. **cypress-mock-server:**

- Umožňuje simulaci backendového serveru pomocí lokálního mock serveru.

xiv. **cypress-log-to-output:**

- Přesměrovává logy Cypressu do terminálu či výstupního souboru – vhodné v CI prostředích.

xv. **cypress-testrail:**

- Integruje Cypress s TestRail pro automatizaci aktualizací testovacích případů.

xvi. **cypress-react-selector:**

- Umožňuje vybírat React komponenty podle názvů, což je intuitivní pro testování React aplikací.

xvii. **cypress-drag-drop:**

- Přidává příkazy pro simulaci drag-and-drop akcí v prohlížeči.

xviii. **cypress-iframe:**

- Ulehčuje testování iframe prvků poskytováním příkazů pro interakci s jejich obsahem.

xix. **cypress-stub:**

- Poskytuje utility pro stubování a špehování funkcí – užitečné při unit testingu a izolaci chování.

xx. **cypress-visual-regression:**

- Integruje vizuální regresní testování, které umožňuje detekovat vizuální změny v aplikaci.

## B. Rozšíření

### 1. Zvyšování možností Cypressu pomocí rozšíření:

- **Definice:**

Rozšíření mohou označovat jak pluginy, tak další JavaScriptové moduly, které dále rozšiřují možnosti Cypressu.

- **Vlastní pluginy:**

Můžete napsat vlastní pluginy pro řešení specifických testovacích potřeb (např. vlastní logování, integrace s CI systémem).

- **Příklady:**

- Vytvoření vlastní úlohy pro čtení nebo zápis do souboru.
- Rozšiřování Cypress příkazů za účelem vytvoření vyšší abstrakce pro opakované UI interakce.

## C. Osvědčené postupy

### 1. Výběr vhodných pluginů:

- **Vyvarujte se zbytečného přebytku:**

Zvolte pouze pluginy, které přidávají jasnou hodnotu do testovací sady. Zbytečné pluginy mohou zpomalit testování a komplikovat údržbu.

- **Zvažte údržbu a podporu komunity:**

Používejte pluginy, které jsou aktivně udržované a mají silnou komunitu, což zajistí kompatibilitu s novými verzemi Cypressu.

### 2. Údržba konfigurací pluginů:

- **Centralizace konfigurací:**

Uchovávejte konfigurace pluginů ve vašem `cypress.config.js` nebo v dedikovaných konfiguračních souborech, aby byla zajištěna konzistentnost.

- **Dokumentujte změny:**

Sledujte verze pluginů a změny v konfiguraci v dokumentaci nebo komentářích ve verzovacím systému.

- **Pravidelné revize:**

Pravidelně kontrolujte seznam pluginů a odstraňujte ty nepotřebné či aktualizujte zastaralé.

## D. Aktivita

### 1. Instalace a konfigurace pluginu Cypress Axe:

- **Aktivita:**

Nainstalujte plugin jako `cypress-axe` do svého ukázkového projektu.

- **Kroky:**

- Instalace přes npm.
- Import pluginu v support souboru.
- Napsání jednoduchého testu, který zavede axe a otestuje porušení pravidel přístupnosti.

- **Příklad ukázky kódu:**

```
// cypress/support/commands.js nebo index.js
import 'cypress-axe';

// V testovacím souboru:
describe('Testování přístupnosti s cypress-axe', () => {
  beforeEach(() => {
    cy.visit('/');
    cy.injectAxe();
  });

  it('by nemělo být zjištěno žádné porušení přístupnosti na domovské stránce', () => {
    cy.checkA11y();
  });
});
```

### 2. Instalace a konfigurace pluginu Cypress Cucumber:

#### 1. Feature File (login.feature)

Vytvořte soubor (např. `cypress/e2e/login.feature`) s tímto obsahem:

## Feature: User Login

As a registered user

I want to log in to the application

So that I can access my dashboard

### Scenario: Successful login with valid credentials

Given I open the login page

When I enter valid credentials

And I submit the login form

Then I should see a welcome message

### Vysvětlení:

- Sekce **Feature** poskytuje přehled uživatelského příběhu.
- Sekce **Scenario** popisuje konkrétní případ: úspěšné přihlášení.
- **Kroky** (Given, When, And, Then) vyjadřují chování v běžném jazyce.

## 2. Step Definitions (login.steps.js)

Vytvořte soubor definic kroků (např. `cypress/e2e/login.steps.js` ) s tímto obsahem:

```
import { Given, When, Then } from '@badeball/cypress-cucumber-preprocessor';

Given('I open the login page', () => {
  cy.visit('/login'); // Předpokládá, že baseUrl je nastavena v cypress.config.js
});

When('I enter valid credentials', () => {
  // Nahradte skutečnými selektory a platnými údaji
  cy.get('[data-testid="login-username-input"]').type('demoUser');
  cy.get('[data-testid="login-password-input"]').type('demoPass');
});

When('I submit the login form', () => {
  cy.get('[data-testid="login-submit-button"]').click();
});

Then('I should see a welcome message', () => {
  cy.get('[data-testid="login-success-message"]')
    .should('be.visible')
    .and('contain', 'Welcome'); // Přizpůsobte dle skutečné zprávy aplikace
});
```

### Vysvětlení:

- Definice kroků používají funkce `Given`, `When` a `Then` importované z preprocesoru.
- Každý krok odpovídá textu z feature souboru.
- Příkazy uvnitř každého kroku simulují uživatelské interakce a ověřují je pomocí Cypressu.

## 3. Nastavení cypress-cucumber-preprocessor

### i. Instalace:

Nainstalujte potřebné balíčky:

```
npm install --save-dev @badeball/cypress-cucumber-preprocessor
npm install --save-dev @bahmutov/cypress-esbuild-preprocessor
```

### ii. Konfigurace:

Aktualizujte svůj `cypress.config.js`, abyste integrovali preprocesor:



```

import { defineConfig } from 'cypress';
import createBundler from "@bahmutov/cypress-esbuild-preprocessor"
import { addCucumberPreprocessorPlugin } from "@badeball/cypress-cucumber-preprocessor"
import createEsbuildPlugin from "@badeball/cypress-cucumber-preprocessor/esbuild"

export default defineConfig({
  e2e: {
    specPattern: '**/*.feature', // Ujistěte se, že vaše feature soubory jsou zpracovány
    async setupNodeEvents(on, config) {
      // Inicializace preprocesoru
      await addCucumberPreprocessorPlugin(on, config)
      on(
        "file:preprocessor",
        createBundler({
          plugins: [createEsbuildPlugin(config)],
        })
      )
      return config;
    },
  },
});

```

### iii. Spuštění testů:

Spustíte testy jako obvykle:

```
npx cypress open
```

Tím se zobrazí vaše feature soubory v Cypress Test Runneru a můžete je spouštět jako běžné end-to-end testy.

## Shrnutí

- **BDD s Cypressem:**

Příklad výše ukazuje, jak psát BDD testy pomocí Gherkin syntaxe ve feature souboru a následně implementovat kroky v JavaScriptu.

- **Výhody:**

- Zlepšuje čitelnost testů.
- Přemostňuje mezeru mezi netechnickými zúčastněnými a vývojáři.
- Pomáhá zajistit soulad testů s obchodními požadavky.

## E. Zdroje

- Dokumentace Cypress Plugins:

[Cypress Plugins](#)

- Dokumentace cypress-axe:

[cypress-axe GitHub Repository](#)

- Plugin cypress-grep:

[cypress-grep GitHub Repository](#)

- Příklady z komunity:

Hledejte open source Cypress projekty, které ukazují pluginy a rozšíření v praxi.

## Možné otázky a odpovědi studentů

1. **Otázka:** *Co jsou Cypress pluginy a proč jsou důležité?*

**Odpověď:** Cypress pluginy jsou moduly, které rozšiřují funkčnost Cypressu. Umožňují přidávat funkce jako testování přístupnosti, nahrávání souborů a další, což může učinit vaši testovací sadu robustnější a komplexnější.

2. **Otázka:** *Jak cypress-axe zlepšuje náš testovací proces?*

**Odpověď:** cypress-axe integruje engine pro testování přístupnosti axe s Cypressem a umožňuje automatizované testy přístupnosti. Díky tomu se vaše aplikace lépe přizpůsobuje standardům přístupnosti bez nutnosti manuální kontroly.

3. **Otázka:** *Jaký je rozdíl mezi pluginem a rozšířením v Cypressu?*

**Odpověď:** Zatímco obojí rozšiřuje funkčnost Cypressu, pluginy jsou obvykle instalované z npm a konfigurované prostřednictvím konfiguračních souborů Cypressu. Rozšíření mohou zahrnovat vlastní kód nebo moduly napsané speciálně pro konkrétní testovací potřeby.

4. **Otázka:** *Proč bychom se měli vyhýbat příliš velkému množství pluginů?*

**Odpověď:** Každý plugin představuje určitou zátěž pro vaši testovací sadu. Nadměrné množství pluginů může zpomalit provádění testů a zvýšit složitost údržby. Je důležité vybírat pluginy, které přinášejí jasné výhody a jsou aktivně udržované.

5. **Otázka:** *Co mám dělat, když konfigurace pluginu rozbije mé testy?*

**Odpověď:** Zkontrolujte dokumentaci pluginu, ověřte případné konflikty s existující konfigurací a zkuste plugin izolovat v menším testovacím případě pro nalezení chyby. Je také dobré držet konfigurace pluginů centralizovaně a dobře zdokumentované.